



Performance

Profiling

Profiling is about analysing the execution of a program and measure aggregated data. These data can be the elapsed time for each function, the executed SQL queries...

While profiling does not improve the performance of a program by itself, it can prove very helpful in finding performance issues and identifying which part of the program is responsible for them.

Odoo provides an integrated profiling tool that allows recording all executed queries and stack traces during execution. It can be used to profile either a set of requests of a user session, or a specific portion of code. Profiling results can be either inspected with the integrated [speedscope](#)   open source app allowing to visualize a flamegraph view or analyzed with custom tools by first saving them in a JSON file or in the database.

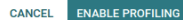
Enable the profiler

The profiler can either be enabled from the user interface, which is the easiest way to do so but allows profiling only web requests, or from Python code, which allows profiling any piece of code including tests.

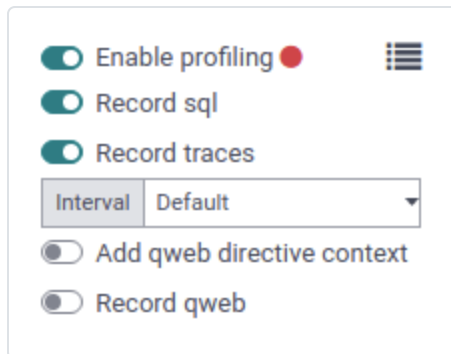
Enable from the user interface

Enable from Python code

1. [Enable the developer mode](#).
2. Before starting a profiling session, the profiler must be enabled globally on the database. This can be done in two ways:
 - Open the [developer mode tools](#), then toggle the **Enable profiling** button. A wizard suggests a set of expiry times for the profiling. Click on **ENABLE PROFILING** to enable the profiler globally.

A screenshot of a button labeled 'ENABLE PROFILING' in a dark teal color, with a 'CANCEL' button to its left.

- Go to **Settings** → **General Settings** → **Performance** and set the desired time to the field **Enable profiling until**.
3. After the profiler is enabled on the database, users can enable it on their session. To do so, toggle the **Enable profiling** button in the **developer mode tools** again. By default, the recommended options **Record sql** and **Record traces** are enabled. To learn more about the different options, head over to **Collectors**.



When the profiler is enabled, all the requests made to the server are profiled and saved into an `ir.profile` record. Such records are grouped into the current profiling session which spans from when the profiler was enabled until it is disabled.

Note

Odoo Online databases cannot be profiled.

Analyse the results

To browse the profiling results, make sure that the **profiler is enabled globally on the database**, then open the **developer mode tools** and click on the button in the top-right corner of the profiling section. A list view of the `ir.profile` records grouped by profiling session opens.

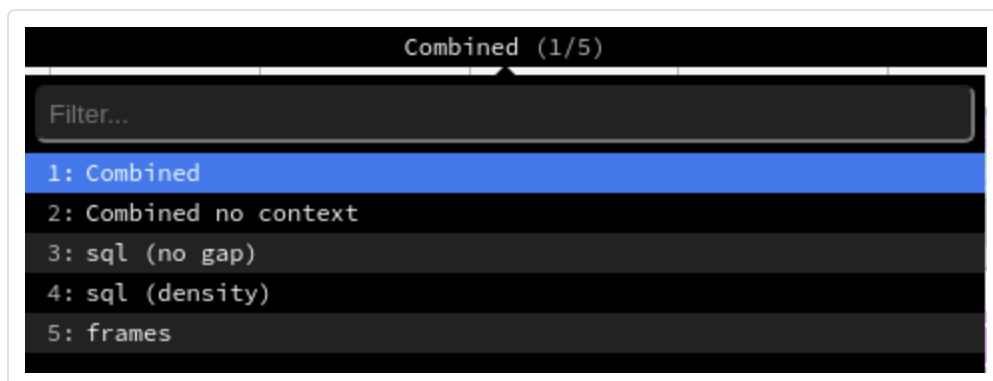
☐	10/08/2021 12:32:40	2021-10-08 10:32:37 Mitchell Admin	/web/dataset/call_kw/res.users/systray_get_activities?	24	/web/speedscope/24	0.02
☐	10/08/2021 12:32:40	2021-10-08 10:32:37 Mitchell Admin	/web/dataset/call_kw/res.users/has_group?	4	/web/speedscope/23	0.01
☐	10/08/2021 12:32:40	2021-10-08 10:32:37 Mitchell Admin	/web/dataset/call_kw/voip.phonecall/get_missed_call_info?	7	/web/speedscope/22	0.01
☐	10/08/2021 12:32:40	2021-10-08 10:32:37 Mitchell Admin	/mail/init_messaging?	64	/web/speedscope/21	0.10
☐	10/08/2021 12:32:40	2021-10-08 10:32:37 Mitchell Admin	/web/dataset/call_kw/voip.configurator/get_pbx_config?	4	/web/speedscope/20	0.01
☐	10/08/2021 12:32:40	2021-10-08 10:32:37 Mitchell Admin	/web/dataset/call_kw/res.users/has_group?	3	/web/speedscope/19	0.01
☐	10/08/2021 12:32:40	2021-10-08 10:32:37 Mitchell Admin	/web/webclient/load_menus/6670d2cb2111c89f245fcd1c1fc8c121e803c9fce4c269353a8046e2948835e6?	7	/web/speedscope/18	0.04
☐	10/08/2021 12:32:39	2021-10-08 10:32:37 Mitchell Admin	/web?debug=1	124	/web/speedscope/17	0.37

Each record has a clickable link that opens the speedscope results in a new tab.



Speedscope falls out of the scope of this documentation but there are a lot of tools to try: search, highlight of similar frames, zoom on frame, timeline, left heavy, sandwich view...

Depending on the profiling options that were activated, Odoo generates different view modes that you can access from the top menu.



- The **Combined** view shows all the SQL queries and traces merged together.

another, without any Python logic. This is useful for optimizing SQL only.

- The **sql (density)** view shows only all the SQL queries, leaving gap between them. This can be useful to spot if either SQL or Python code is the problem, and to identify zones in where many small queries could be batched.
- The **frames** view shows the results of only the **periodic collector**.

Important

Even though the profiler has been designed to be as light as possible, it can still impact performance, especially when using the **Sync collector**. Keep that in mind when analyzing speedscope results.

Collectors

Whereas the profiler is about the *when* of profiling, the collectors take care of the *what*.

Each collector specializes in collecting profiling data in its own format and manner. They can be individually enabled from the user interface through their dedicated toggle button in the **developer mode tools**, or from Python code through their key or class.

There are currently four collectors available in Odoo:

Name	Toggle button	Python key	Python class
SQL collector	Record sql	sql	SqlCollector
Periodic collector	Record traces	traces_async	PeriodicCollector
QWeb collector	Record qweb	qweb	QwebCollector
Sync collector	No	traces_sync	SyncCollector

By default, the profiler enables the SQL and the Periodic collectors. Both when it is enabled from the user interface or Python code.

SQL collector

The SQL collector saves all the SQL queries made to the database in the current thread (for all cursors), as well as the stack trace. The overhead of the collector is added to the analysed

the combined speedscope view.

```
class odoo.tools.profiler.SQLCollector \[source\] 
```

Saves all executed queries in the current thread with the call stack.

Periodic collector

This collector runs in a separate thread and saves the stack trace of the analysed thread at every interval. The interval (by default 10 ms) can be defined through the **Interval** option in the user interface, or the `interval` parameter in Python code.

Warning

If the interval is set at a very low value, profiling long requests will generate memory issues. If the interval is set at a very high value, information on short function executions will be lost.

It is one of the best way to analyse performance as it should have a very low impact on the execution time thanks to its separate thread.

```
class odoo.tools.profiler.PeriodicCollector (interval=None) \[source\] 
```

QWeb collector

This collector saves the Python execution time and queries of all directives. As for the [SQL collector](#), the overhead can be important when executing a lot of small directives. The results are different from other collectors in terms of collected data, and can be analysed from the `ir.profile` form view using a custom widget.

It is mainly useful for optimizing views.

```
class odoo.tools.profiler.QwebCollector \[source\] 
```

Record qweb execution with directive trace.

Sync collector

This collector saves the stack for every function's call and return and runs on the same thread, which greatly impacts performance.

Record complete execution synchronously. Note that `--limit-memory-hard` may need to be increased when launching Odoo.

Performance pitfalls

- Be careful with randomness. Multiple executions may lead to different results. E.g., a garbage collector being triggered during execution.
- Be careful with blocking calls. In some cases, external `c_call` may take some time before releasing the GIL, thus leading to unexpected long frames with the **Periodic collector**. This should be detected by the profiler and give a warning. It is possible to trigger the profiler manually before such calls if needed.
- Pay attention to the cache. Profiling before that the `view / assets /...` are in cache can lead to different results.
- Be aware of the profiler's overhead. The **SQL collector**'s overhead can be important when a lot of small queries are executed. Profiling is practical to spot a problem but you may want to disable the profiler in order to measure the real impact of a code change.
- Profiling results can be memory intensive. In some cases (e.g., profiling an install or a long request), it is possible that you reach memory limit, especially when rendering the speedscope results, which can lead to an HTTP 500 error. In this case, you may need to start the server with a higher memory limit: `--limit-memory-hard $((8*1024**3))`.

Good practices

Batch operations

When working with recordsets, it is almost always better to batch operations.

Example

Don't call a method that runs SQL queries while looping over a recordset because it will do so for each record of the set.

```
def _compute_count(self):
    for record in self:
        domain = [('related_id', '=', record.id)]
        record.count = other_model.search_count(domain)
```

```
def _compute_count(self):
    domain = [('related_id', 'in', self.ids)]
    counts_data = other_model._read_group(domain, ['related_id'], ['__count'])
    mapped_data = dict(counts_data)
    for record in self:
        record.count = mapped_data.get(record, 0)
```

Note

This example is not optimal nor correct in all cases. It is only a substitute for a `search_count`. Another solution could be to prefetch and count the inverse `One2many` field.

Example

Don't create records one after another.

```
for name in ['foo', 'bar']:
    model.create({'name': name})
```

Instead, accumulate the create values and call the `create` method on the batch. Doing so has mostly no impact and helps the framework optimize fields computation.

```
create_values = []
for name in ['foo', 'bar']:
    create_values.append({'name': name})
records = model.create(create_values)
```

Example

Fail to prefetch the fields of a recordset while browsing a single record inside a loop.

```
for record_id in record_ids:
    model.browse(record_id)
```

```
records = model.browse(record_ids)
for record in records:
    record.foo # One query is executed for the entire recordset.
```

We can verify that the records are prefetched in batch by reading the field `prefetch_ids` which includes each of the record ids. browsing all records together is impractical, If needed, the `with_prefetch` method can be used to disable batch prefetching:

```
for values in values_list:
    message = self.browse(values['id']).with_prefetch(self.ids)
```

Reduce the algorithmic complexity

Algorithmic complexity is a measure of how long an algorithm would take to complete in regard to the size n of the input. When the complexity is high, the execution time can grow quickly as the input becomes larger. In some cases, the algorithmic complexity can be reduced by preparing the input's data correctly.

Example

For a given problem, let's consider a naive algorithm crafted with two nested loops for which the complexity is in $O(n^2)$.

```
for record in self:
    for result in results:
        if result['id'] == record.id:
            record.foo = result['foo']
            break
```

Assuming that all results have a different id, we can prepare the data to reduce the complexity.

```
mapped_result = {result['id']: result['foo'] for result in results}
for record in self:
```


Example

Choosing the bad data structure to hold the input can lead to quadratic complexity.

```
invalid_ids = self.search(domain).ids
for record in self:
    if record.id in invalid_ids:
        ...
```

If `invalid_ids` is a list-like data structure, the complexity of the algorithm may be quadratic.

Instead, prefer using set operations like casting `invalid_ids` to a set.

```
invalid_ids = set(invalid_ids)
for record in self:
    if record.id in invalid_ids:
        ...
```

Depending on the input, recordset operations can also be used.

```
invalid_ids = self.search(domain)
for record in self - invalid_ids:
    ...
```

Use indexes

Database indexes can help fasten search operations, be it from a search in the or through the user interface.

```
name = fields.Char(string="Name", index=True)
```

Warning



--	--

 Get Help

- Contact Support
- Ask the Odoo Community

